

UNIMORE-FIM INFORMATICA

Architettura dei Calcolatori

Appunti

Lorenzo Balugani (@EvilBalu)

Secondo semestre

INTRODUZIONE AL CORSO

SVILUPPO E EVOLUZIONE

L'evoluzione dei calcolatori è stata portata avanti da 8 grandi idee:

- Progettazione per il futuro (Legge di Moore)
 - Scalabilità
- Astrazione
 - Aiuta i progettisti a isolare i problemi e affrontarli separatamente
- Rendere il caso più comune d'uso il più veloce
- Ottenere prestazioni elevate mediante...
 - Parallelismo
 - Unità in parallele che lavorano all'unisono
 - Pipelining
 - Parallelizzazione dei flussi di dati (concetto da catena di montaggio)
 - Predizione
- Gerarchizzazione della memoria
- Ridondanza

SOTTO AL PROGRAMMA

Sotto all'applicazione, esistono altri livelli che permettono a quest'ultima di comunicare con l'hardware: immediatamente sotto si trova il layer del sistema operativo, che gestisce le librerie, la gestione della memoria, le risorse in generale e l'I/O, e sotto di esso si trova l'hardware. Esistono vari tipi di codice:

- Codice ad alto livello
- Assembly (il codice ad alto livello lo diventa mediante il compilatore)
- Rappresentazione in bit (l'assembly lo diventa mediante l'assembler)

PRESTAZIONI

La prestazione di un programma è determinata dal **software** e dall'**hardware**:

- **Algoritmo risolutivo**
- **Linguaggio, compilatore, architettura**
- **Processore e memoria**
- **Velocità I/O**

DENTRO AL PROCESSORE

Dentro al processore si trovano:

- I Datapath, unità che eseguono operazioni sui dati
- Control, unità che gestisce le altre unità interne
- Memoria cache di primo livello, un tipo di memoria SRAM estremamente veloce e piccola
- ...

Un processore è caratterizzato dal set di istruzioni in grado di comprendere, e dall'ISA che viene quindi impiegato, ovvero da quale interfaccia hw/sw.

L'attuale processo di creazione di un microprocessore è così definita:

- Taglio del lingotto di silicio in fogli spessi 2 mm (wafer)
- Ottenimento di wafer con sopra i processori finiti
- Lettura e verifica dei wafer, taglio dei wafer in quadratini (die)
- I dice non difettosi vengono "impacchettati" e riverificati

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \frac{\text{Wafer area}}{\text{Die area}}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}/2))^2}$$

Lo Yield è la proporzione tra i dice funzionanti e quelli non funzionanti. Il prezzo di un circuito integrato aumenta con l'aumento delle dimensioni del chip, quindi un processo produttivo che permette dimensioni inferiori del die è più conveniente.

Esistono diversi parametri per definire la performance di un processore:

- Tempo di risposta
- Throughput
 - Lavoro fatto per unità di tempo
- Come queste due unità di misura sono influenzate dal
 - Rimpiazzare il processore
 - Aggiungere processori

TEMPO DI RISPOSTA

Esaminiamo il tempo di risposta. Definiamo $Perf = \frac{1}{Extime}$. Quando diciamo che X è n volte più veloce di Y, diciamo

quindi che $\frac{Perf_x}{Perf_y} = \frac{Extime_y}{Extime_x}$.

Il tempo trascorso è costituito dal tempo totale di risposta, includendo tutti gli aspetti del processo (tempi I/O, tempo di calcolo...) che determina la performance della macchina nel suo complesso, mentre il CPU time è solo il tempo nel quale la macchina è impegnata a calcolare. Come si calcola questo tempo? Conoscendo il periodo del clock e la frequenza del clock (Clock Rate), si dice che $CPUtime = CPUCycles * CPUcycleTime = \frac{ClockCycles}{ClockRate}$.

La performance è quindi migliorata riducendo i cicli di clock, aumentando il clock rate, il che porta i programmatori a cercare un compromesso tra clock rate e numero di cicli.

Un metodo alternativo per valutare il CPU time utilizza l'Instruction Count per il programma e il CPI (ovvero il costo per istruzione) delle istruzioni. L'IC è determinato dal programma, dall'ISA e dal compilatore, mentre i cicli medi per istruzione sono determinati dal processore e, se le istruzioni hanno un diverso CPI, dal mix di istruzioni aventi CPI diversi (si fa una sorta di media).

$$\begin{aligned} \text{Clock Cycles} &= \text{Instruction Count} \times \text{Cycles per Instruction} \\ \text{CPU Time} &= \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} \\ &= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}} \end{aligned}$$

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i) \quad \text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right) \quad \text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

Relative frequency

La performance dipende quindi:

- Dall'algoritmo (IC, CPI)
- Linguaggio di programmazione (IC, CPI)
- Compilatore (IC, CPI)
- ISA (IC, CPI T_c)

Le reti logiche rappresentano un livello di astrazione che studia i sistemi digitali a livello di componenti logici

elementari indipendentemente dalla tecnologia usata. Una rete logica è un sistema digitale dagli n segnali binari d'ingresso e da m segnali binari di uscita.

PROPRIETÀ DELLE RETI LOGICHE

- **Interconnessione:** l'interconnessione di più reti logiche, aventi per ingresso segnali esterni o uscite di altre reti logiche e per uscite segnali di uscita esterni o ingressi di altre reti logiche è ancora una rete logica.
- **Decomposizione:** Una rete logica complessa può essere decomposta in unità logiche più semplici
- **Decomposizione in parallelo**

TIPI DI RETI LOGICHE

- **Reti logiche combinatorie:** l'output è stabilito solo dagli ingressi
- **Reti logiche sequenziali:** l'output è stabilito anche dagli input passati

Le reti combinatorie quindi non hanno stato, mentre quelle sequenziali lo posseggono (hanno memoria). Per sapere l'uscita di una rete logica sequenziale posso o ricordarmi tutti gli ingressi dati alla rete dalla sua accensione, oppure memorizzo uno stato del sistema, che riassume tutti gli ingressi precedenti per valutare le uscite.

DESCRIZIONE RETI COMBINATORIA

1. A parole (poco formale e precisa)
2. Tabelle di verità
3. Mappe
4. Espressioni algebra Booleana
5. Schema logico
6. Forme d'onda
7. Linguaggi descrizione hardware

TABELLA DI VERITÀ

La tabella di verità è una tabella che associa tutte le possibili combinazioni degli ingressi alle corrispondenti configurazioni degli ingressi alle corrispondenti configurazioni delle uscite e indica esaustivamente il comportamento della rete logica. Se la rete ha n ingressi e m uscite, la tabella avrà $m + n$ colonne e 2^n uscite. Una tabella si dice completamente specificata se ogni valore della tabella assume il valore logico 1/0, mentre si dicono non completamente specificate se ci sono condizioni di indifferenza.

GATE ELEMENTARI

Ingresso	Massa	Filo	Not	Vcc
0	0	0	1	1
1	0	1	0	1

A	B	And	Or	Nand	Nor
0	0	0	0	1	1
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	1	0	0

ALGEBRA DI BOOLE

Definizione: Funzione booleana

Una funzione completamente specificata di n variabili è l'insieme di tutte le possibili coppie formate da un elemento del dominio e del codominio. Un metodo standard per descrivere una funzione booleana è quello di usare una tabella di verità.

Nell'algebra booleana, il valore complementato di A si indica con $\overline{A}$.

Definizione: Espressione booleana

Un'espressione secondo l'algebra booleana è una stringa di elementi di B che soddisfa le seguenti regole:

- Una costante è un'espressione
- Una variabile è un'espressione
- Se X è espressione allora lo è anche il suo complemento
- Se X e Y sono espressioni allora lo è anche il loro prodotto e la sua somma

Teorema

Ogni espressione di n variabili descrive una funzione completamente specificata che può essere valutata attribuendo ad ogni variabile un valore assegnato.

Teorema

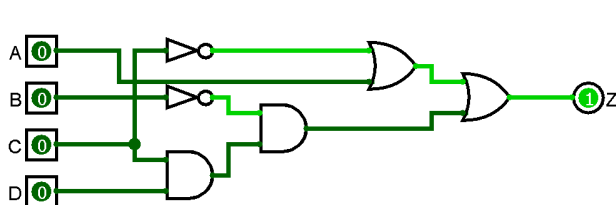
Una espressione di n variabili descrive in maniera univoca uno schema logico di AND, OR e NOT.

METODO DELLA PRIMA FORMA CANONICA

Il metodo consente, da una tabella di verità, di ricostruire l'espressione logica:

1. Trova le righe della tabella in cui y ha valore alto;
2. Si scrive il prodotto di tutte le variabili di quelle righe, negando le variabili uguali a 0 e si sommano i prodotti;
3. Si semplifica

ESEMPIO DI ANALISI



$$z = z_1 + z_2, z_1 = a + \overline{c}, z_2 = \overline{b} * z_3, z_3 = c * d$$
$$z = (a + \overline{c}) + \overline{b} * (c * d)$$

TEOREMI DELL'ALGEBRA DI BOOLE

Teorema	Espressione	Espressione duale
Identità	$A+0=A$	$A*1=A$
Complemento	$A+!A=1$	$A*!A=0$
Involuzione	$!!A=A$	
Idem Potenza	$A+A=A$	$A*A=A$
Annullamento	$A+1=1$	$A*0=0$
Assorbimento	$A+AB=A$	$A*(A+B)=A$
Semplificazione	$A+!AB=A+B$	$A*(!A+B)=A*B$
Adiacenza	$AB+A!B=A$	$(A+B)*(A+!B)=A$
Consenso	$AB+BC+!AC=AB+!AC$	$(A+B)*(B+C)*(!A+C)=(A+B)*(!A+C)$
De morgan	$!(A+B)=!A*!B$	$!(A*B)=!A+!B$

Valgono anche la commutativa, l'associativa e la distributiva.

BIT DI PARITÀ

Al fine di trovare errori durante la trasmissione dei dati vengono impiegati i codici rilevatori di errore. Questi possono essere ridondanti, in cui l'insieme dei simboli dell'alfabeto è minore dell'insieme di configurazioni rappresentabili con il codice, oppure con i bit di parità, in cui viene aggiunto un bit alla sequenza di bit. La parità può essere pari (l'1 aggiunto rende pari il numero di 1 nella sequenza), mentre quella dispari è l'esatto opposto. In un sistema nel quale viene controllata la parità pari, se viene trovata una parità dispari sappiamo che si è verificato un errore.

FORMA CANONICA PS (SECONDA FORMA CANONICA)

La forma canonica del prodotto di somme indica come una funzione di n variabili può essere rappresentata in un solo modo, ovvero come prodotto di somme di n variabili detti maxtermini. I maxtermini sono la somma logica di n letterali ognuno dei quali compare in forma vera o complementata ma mai in entrambe.

MAPPA DI KARNAUGH

Una mappa di Karnaugh è una mappa in cui le configurazioni successive in ogni lato sono adiacenti, con due configurazioni che sono adiacenti logicamente se differiscono di un solo bit sono adiacenti geometricamente se corrispondono a configurazioni adiacenti (adiacenza geometrica e logica, nelle mappe di Karnaugh, combaciano).

Criteri geometrici di adiacenza:

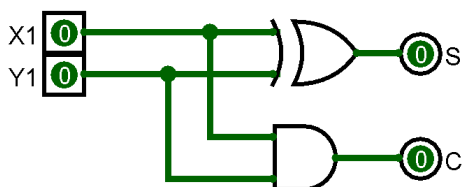
- 1 lato in comune
- Estremità di riga o colonna
- Stessa posizione in sottomatrici adiacenti

Ogni casella della mappa è adiacente a caselle corrispondenti a mintermini (maxtermini) aventi distanza di hamming unitaria dal mintermine (maxtermine) corrispondente alla casella considerata.

Per i raggruppamenti, consultare le slide.

RETI LOGICHE DI MEDIA COMPLESSITÀ

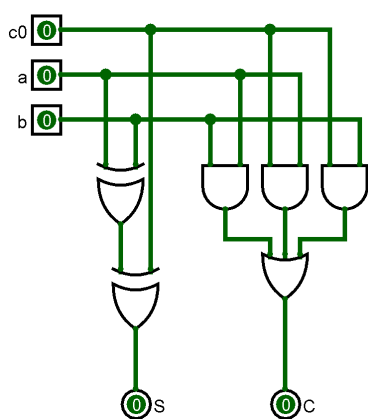
HALF ADDER



Sommatore con due ingressi, un'uscita per il risultato e una per il resto.

X1	Y1	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

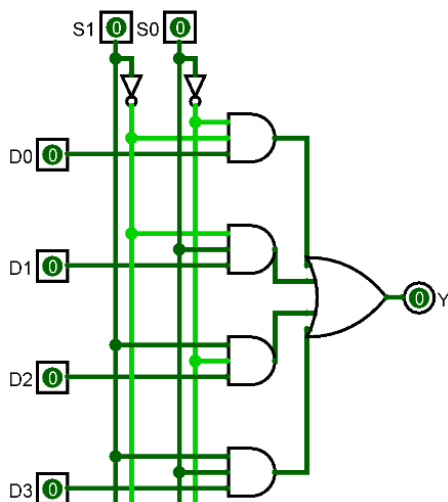
FULL ADDER



c0	a	b	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sommatore completo con ingresso per riporto. Più full adder possono essere concatenati per generare un sommatore a bit superiori.

MULTIPLEXER



Selettore di ingressi che sfrutta il concetto di “gate” delle porte logiche (nel senso che l’AND sbarra). Data una certa combinazione degli S1, S0 viene fatto passare un determinato segnale.

S1	S0	D0	D1	D2	D3	Y
0	0	0	0	0	0	0
0	0	0	0	0	1	0
0	0	0	0	1	0	0
0	0	0	0	1	1	0
0	0	0	1	0	0	0
0	0	0	1	0	1	0
0	0	0	1	1	0	0
0	0	0	1	1	1	0
0	0	1	0	0	0	1
0	0	1	0	0	1	1
0	0	1	0	1	0	1
0	0	1	0	1	1	1
0	0	1	1	0	0	1

Il demultiplexer esegue il compito inverso: sposta su un’uscita preselezionata l’unico segnale in ingresso.

BUFFER A TRE STATI

I circuiti logici a terzo stato sono circuiti che prevedano un terzo stato detto di alta impedenza, ovvero nel quale l’uscita si comporta come se fosse “scollegata” dal resto della rete.

Il buffer three-state è quindi dotato di un ingresso aggiuntivo, chiamato generalmente c, che imposta uno stato di alta impedenza

UNITÀ ARITMETICO-LOGICA

L’ALU è un circuito combinatorio in grado di eseguire operazioni aritmetico-logiche, situata all’interno del processore.

RETI SEQUENZIALI

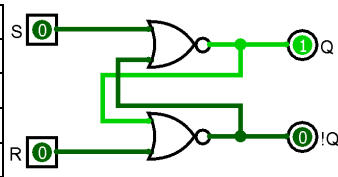
Una rete sequenziale è una rete logica in cui in ogni istante le uscite e il comportamento interno dipendono non solo dalla configurazione degli ingressi in quell’istante ma anche da quelli negli stati precedenti: il comportamento dipende quindi anche dalla storia passata, con le variazioni degli ingressi che modificano anche lo stato interno. Una rete sequenziale può essere asincrona, se le variazioni vengono sentite in un qualsiasi istante, oppure sincrona, nel caso in cui siano sentite in presenza di un segnale di sincronizzazione (il clock, ovvero un segnale free-running generato da un circuito che ha la caratteristica di essere periodico). Di questo segnale possono interessare i livelli oppure il fronte. Le reti che vedremo di seguito sono attivate in base al fronte (edge-triggered).

LATCH

LATCH SR

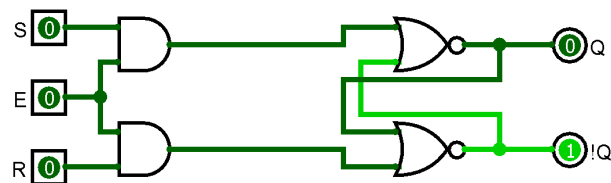
SENZA ABILITAZIONE

S	R	Q(T)	!Q(t)
0	0	Q(t-1)	!Q(t-1)
0	1	0	1
1	0	1	0
1	1	-	-



Su una memoria bistabile SR si possono fare due tipi di operazione: SET (lo porto a 1) e RESET (lo porto a 0). La situazione 00 è equivalente all'hold, ovvero l'uscita del circuito dipende da quello che è in memoria.

SENSIBILE A LIVELLO



Quando non abilitato, il latch sr va in automatico in posizione hold.

LATCH D

D	CK	Q(T)	!Q(t)	
-	0	Q(t-1)	!Q(t-1)	
0	1	0	1	Reset
1	1	1	0	Set

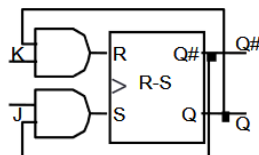
Memoria che mantiene l'uscita costante se il clock non è attivo e cambia il segnale interno in base al segnale D. E' un latch SR abilitazione con D che entra in S ed entra in R negato.

FLIP FLOP

I latch sono dei bistabili definiti come trasparenti e in grado di memorizzare segnali in funzione di un segnale di sincronizzazione, e la transizione avviene per tutto il tempo in cui il clock è attivo (alto). I flip flop, invece, sono sempre dei bistabili, ma privi della trasparenza. Questo vuol dire che il cambiamento di uscita non è conseguenza del cambiamento dell'ingresso di un dato ma è conseguenza del cambiamento (edge triggered) di un ingresso di controllo (sincrono o asincrono), che memorizza l'ingresso attuale.

FLIP FLOP JK

J	K	Q	!Q	Descrizione
0	0	Q	!Q	HOLD
0	1	0	1	Reset
1	0	1	0	Set
1	1	!Q	Q	Toggle



Una variazione del flip flop JK è il flip flop T, che ha un solo segnale T nei due ingressi del flip flop (fa solo il toggle).

ASPETTI SIGNIFICATIVI DEL CLOCK

Periodo di clock: lunghezza del ciclo

Frequenza del clock: 1/periodo

Duty Cycle: tempo in cui il clock è alto

Tsetup: tempo nel quale gli ingressi devono rimanere stabili prima del fronte del clock per essere memorizzati

Thold: tempo in cui gli ingressi devono rimanere stabili dopo l'evento del clock

REGISTRI

Un registro è un elemento di memoria in cui n flip flop sono controllati dal medesimo segnale di clock creando un'unità in grado di memorizzare un'informazione di n bit. Un registro è generalmente corredato di un segnale di abilitazione ingressi e di un segnale di abilitazione uscite. Un registro è quindi un tipo di memoria, ovvero un dispositivo in grado di immagazzinare dati con ogni unità di memorizzazione chiamata cella di memoria, identificata da un indirizzo.

AUTOMA A STATI FINITI

Una rete sequenziale, come abbiamo detto, memorizza le informazioni degli ingressi che si verificano nel tempo, che avviene in stati interni con le variabili che definiscono lo stato interno sono memorizzate in elementi di retroazione. Tra le reti sequenziali hanno grande importanza le macchine a stati finiti, in cui gli elementi di retroazione sono flip flop con unico segnale di clock: l'insieme dei flip flop è detto registro di stato. Esistono due modelli per descrivere una macchina a stati finiti:

- Automa di Mealy, il valore delle uscite dipende dallo stato presente e dagli ingressi in quell'istante
- Automa di Moore, il valore delle uscite dipende solo dallo stato presente

Anche il processore è descrivibile come una macchina a stati finiti, con la parte sequenziale composta dalla Control Unit.

Una rete sequenziale è quindi definita da una tupla $\langle X, Z, S, F, G \rangle$ e richiede la realizzazione di due funzioni combinatorie (F,G) che dipendono da due insiemi di valori (X,S). Di solito si realizza l'automa di Moore, in quanto concettualmente più semplice.

SINTESI DI RETI SEQUENZIALI (DA SLIDE 33)

1. Si prepara una descrizione a parole
2. Si definisce il diagramma degli stati per definire le transizioni tra di essi e si crea la tabella di flusso
 - a. Il diagramma degli stati è un grafo con tanti nodi quanti gli stati e tanti archi quante le transizioni da uno stato all'altro. Se il modello è di Mealy, i valori delle uscite sono rappresentati sugli archi, altrimenti nei nodi.
3. Si minimizzano gli stati
4. Dal diagramma minimizzato e dalla tabella di flusso si crea la tabella delle transizioni e delle uscite
5. Implementazione

RISC V

Comando	Sintassi	Spiegazione
add	add a, b, c	Il risultato di b+c va in a
sub	sub a, b, c	Il risultato di b-c va in a
addi	addi a,b, 20	Il risultato di b+20 va in a
ld	ld x5, 40(x6)	Carica in x5 il valore contenuto in x6+40
sd	sd x5, 40(x6)	Salva x5 nella memoria con indirizzo x6+40
and	and x5, x6, x7	Il risultato di x6 & x7 viene salvato in x5
or	or x5, x6, x7	Il risultato di x6 x7 viene salvato in x5
xor	xor x5, x6, x7	Il risultato dell' xor tra x6 e x7 viene salvato in x5
Shift Left	sll x5, x6, x7	Shift a sinistra
Shift Right	slr x5, x6, x7	Shift a destra
beq	beq x5, x6, 100	Se x5 == x6, PC+100
bne	bne x5, x6, 100	Se x5!=x6, PC+100
blt	blt x5, x6, 100	Se x5<x6, PC+100

bge	bge x5, x6, 100	Se $x5 \geq x6$, $PC+100$
jal	jal x1, 100	Salto incondizionato. Salva in x1 il PC prima dell'incremento.

Il RISC V è un ISA (set di istruzioni) sviluppato dall'università di Berkeley open source, caratterizzato dalle seguenti operazioni (ne sono listate solo alcune):

I principi di progettazione di questo ISA sono: 1) La semplicità favorisce la regolarità (la regolarità rende le implementazioni più facili e la semplicità permette performance maggiori a prezzo ridotto), 2) Più è piccola la memoria (in termine di suddivisione) più sarà veloce, 3) Un buon design richiede buoni compromessi (formati delle istruzioni simili, ma separati per uniformità di dimensione). Esistono anche pseudo istruzioni, come mv.

La dimensione della parola è 32-bit, e abbiamo a disposizione 32 registri da 64 bit ciascuno:

Registro	Descrizione
x0	Valore costante 0
x1	Indirizzo di ritorno di una funzione
x2	Puntatore alla cima dello stack
x3	Puntatore alla memoria globale (variabili globali)
x4	Puntatore per informazioni tra processi (?)
x8	Registro di attivazione di una funzione
x5-x7 x28-x31	Registri per variabili temporanee (non preservati tra le chiamate)
x9 x18-x27	Registri salvati (preservati tra le chiamate)
x10-x11	Risultati o argomenti di funzioni
x12-x17	Argomenti delle funzioni

```
Ld    x9, 64(x22)
add   x9, x21, x9
sd    x9, 96(x22)
```

La memoria principale è strutturata in modo tale che ogni indirizzo identifichi un byte da otto bit.

Supponendo che h sia in x21 e che A sia un array come indirizzo base in x22, l'istruzione $A[12]=h+A[8]$ diventa il listato qui a fianco, dove il 64 e il 96 servono per accedere agli posizioni dell'array ($8*8=64$, $8*12=96$).

Il motivo per il quale si usano i registri al posto di lavorare direttamente sulla memoria è che i registri sono più veloci, lavorare sulla memoria richiede di usare operatori di caricamento e salvataggio e il compilatore deve usare il più possibile dei registri.

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Il formato delle istruzioni R del Risc-V è il seguente:

- Funct7: opcode addizionale, codice funzione
- Rs2: secondo registro di ingresso
- Rs1: primo registro di ingresso
- Funct3: opcode affizionale, codice funzione
- Rd: registro di destinazione
- Opcode: codice operazione

Il formato delle istruzioni I del Risc-V è il seguente:

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

- Immediate: operando costante, oppure offset aggiunto all'indirizzo di base
- Rs1: ingresso oppure indirizzo base

Il formato delle istruzioni S del Risc-V è il seguente:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- Immediate: offset da aggiungere all'indirizzo di base

- Rs1: indirizzo di base del registro
- Rs2: numero del registro di provenienza

Tutti i formati sono:

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Definizione: Basic Block

Un basic block è una sequenza di istruzioni nella quale non sono presenti istruzioni di tipo ramificante (a parte alla fine e all'inizio), vengono automaticamente trovate dal compilatore per operazioni di ottimizzazione.

PROCEDURE CALLING

La chiamata ad una procedura richiede determinati passaggi:

1. Posizionare i parametri nei registri dall'x10 all'x17
2. Trasferire il controllo alla procedura
3. Ottenere lo spazio per la procedura
4. Eseguire la procedura
5. Immettere il risultato in un registro
6. Ritornare alla posizione di chiamata originale

fact:

```
addi sp,sp,-16      Save return address and n on stack
sd x1,8(sp)
sd x10,0(sp)
addi x5,x10,-1      x5 = n - 1
bge x5,x0,L1        If n>0 (equiv. to n >= 1), go to L1
addi x10,x0,1        Else, set return value to 1
addi sp,sp,16        Pop stack, don't bother restoring values
jalr x0,0(x1)        Return
```

L1: addi x10,x10,-1

```
jal x1,fact          n = n - 1
addi x6,x10,0         Call fact(n-1)
ld x10,0(sp)         Move result of fact(n - 1) to x6
ld x1,8(sp)          Restore caller's n
addi sp,sp,16         Restore caller's return address
mul x10,x10,x6        Pop stack
jalr x0,0(x1)         Return n * fact(n-1)
```

Per avviare una procedura, si usa il comando jal [posizione istruzione chiamante (reg. x1)], label, per uscirne si usa jalr x0, 0(x1).

Per delle chiamate annidate, la questione è un po' più complicata.

Il layout della memoria è suddiviso in diverse aree:

- Text: codice del programma
- Static data: variabili globali
- Dati dinamici: heap
- Stack

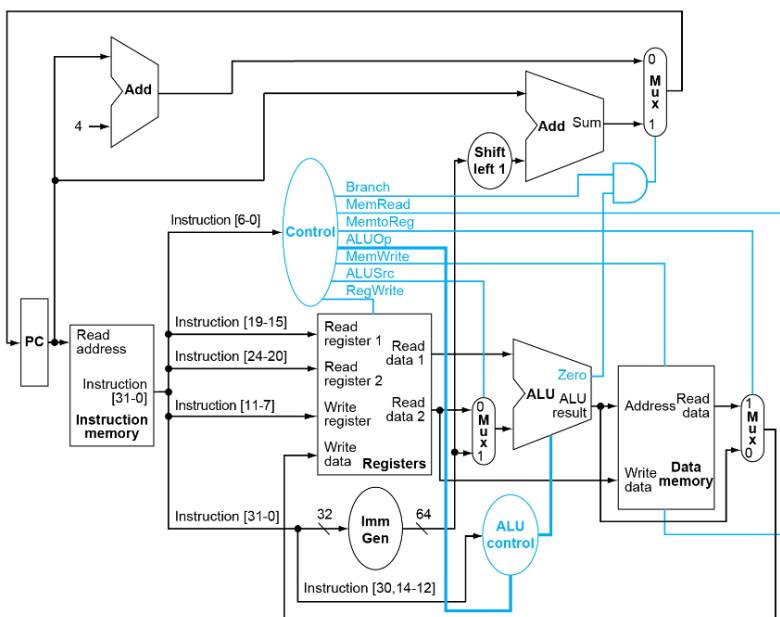
SINCRONIZZAZIONE

Nel momento in cui due processori lavorano sulla medesima area di memoria, se questi non sono sincronizzati si incappa in una race condition. Al fine di ottenere questa sincronia, è necessario un supporto hardware che permetta operazioni di lettura e scrittura atomiche e che non consenta altri accessi a quella parte di memoria quando viene eseguita l'operazione (meccanismo di lock). In Risc-V, le istruzioni per farlo sono lr.d rd, (rs1), che carica dall'indirizzo in rs1 in rd e piazza il flag "riservato" in rs1, e sc.d rd,rs2,(rs1), che salva il valore da rs2 a rs1. Nel caso in cui la locazione è cambiata, ritorna un valore diverso da zero in rd. In questo modo è quindi possibile realizzare un meccanismo che gestisca il lock (dove il lock relativo è contenuto nel registro rd).

Nelle slide, a questo punto, viene eseguito un confronto tra Risc-V, MIPS e l'x86 assembly.

IL PROCESSORE

La performance del processore è determinata dall'Instruction Count (determinati da ISA e compilatore), dal CPI (clock per instruction) e il Cycle time. Per proseguire, definiamo un RISC-V minimale con due istruzioni per memoria (ld, sd), 4 operazioni aritmetico-logiche (add, sub, and, or) e una di control transfer (beq). Il processore, in base al program counter, esegue il fetch delle istruzioni nella memoria delle istruzioni, legge i registri e in base alla classe dell'istruzione la esegue. Il processore, quindi, conterrà il PC, che contiene l'indirizzo dell'istruzione corrente e ne esegue l'incremento quando l'istruzione è stata eseguita (esegue il fetch dell'istruzione dall'Instruction Memory), il register file, che contiene i registri che verranno utilizzati come operandi, l'ALU, che eseguirà le operazioni, e una struttura di controllo, oltre che la memoria principale che nel nostro processore è integrata al suo interno. Il nostro datapath, una volta che tutte le features saranno implementate, avrà questo aspetto:



IMPLEMENTAZIONE DELLA PIPELINE

La realizzazione di una pipeline è equivalente a rendere le risorse del nostro processore utilizzabili in parallelo, in quanto il parallelismo migliora di molto la performance. Esaminando le varie operazioni, notiamo come il caso peggiore richieda 800ps perché tutte le operazioni funzionino correttamente. Con la pipeline, però, riusciamo a ridurre il tutto a 200 ps. Lo speedup che cerco, in questo caso, è un incremento nel throughput. Il RISC-V è progettato per il pipelining (istruzioni tutte a 32 bit, pochi e regolari formati di istruzioni che permettono la decodifica in un solo passo, calcolo dell'indirizzo e operazioni

in memoria al 3° e al 4° step). Esistono situazioni che impediscono l'esecuzione di un'operazione al prossimo ciclo, chiamati Hazard:

- **Hazard Strutturale:** una risorsa è occupata
 - Il load e lo store richiedono l'accesso alla memoria, e il fetch deve stallare in quel ciclo
- **Data Hazard:** necessità di aspettare l'istruzione precedente nell'operazione di lettura/scrittura
 - Add x19, x0, x1
sub x2, x19, x3

Questo si può risolvere in diversi modi

- Forwarding: il risultato non viene immagazzinato nel registro, ma viene prima mandato dove ha bisogno l'istruzione successiva. Questo non è sempre possibile.
- Code scheduling: il codice viene ordinato per evitare degli stalli (si raggruppano i load, ad esempio)

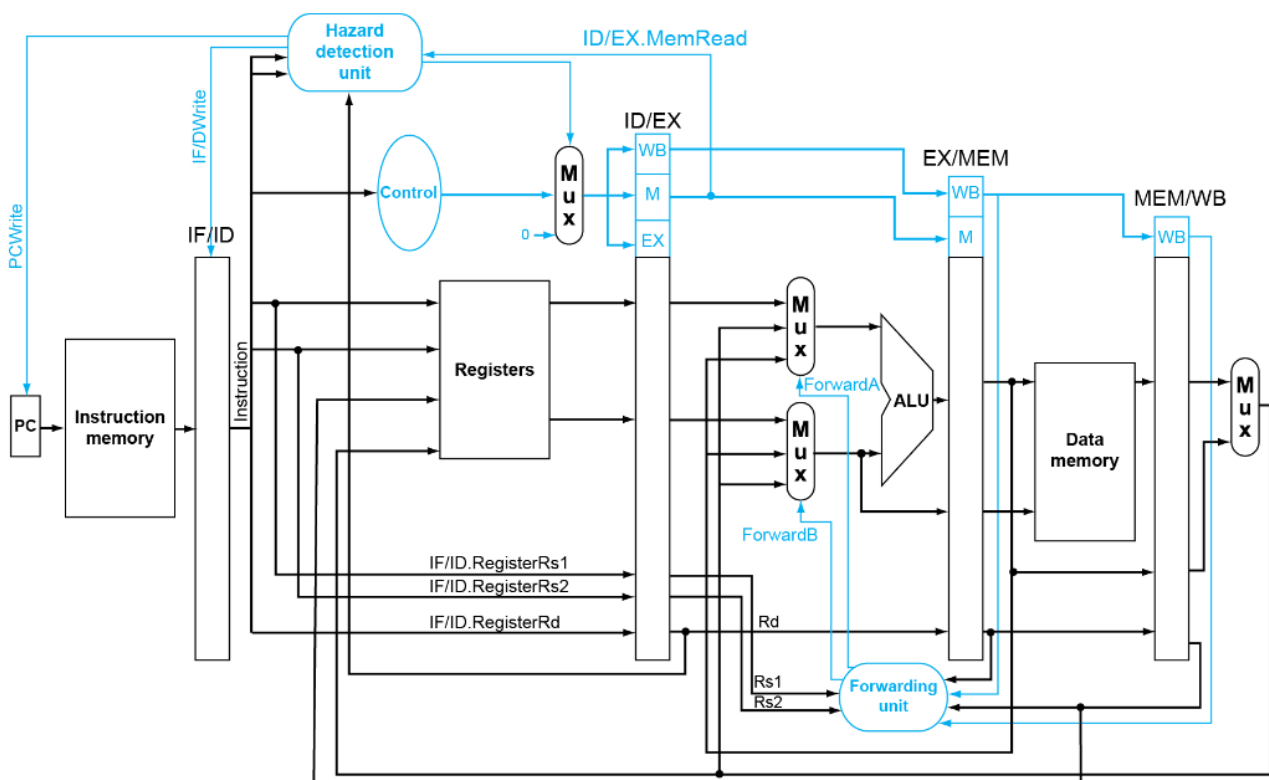
- Un hazard di primo tipo è quando il registro di uscita dall'alu e uno di quelli di ingresso sono i medesimi (il valore di uscita dell'alu, a quel punto, viene messo in ingresso) [Fase EX/MEM]
- Un hazard di secondo tipo è quando il dato si trova in un'altra fase (non quella dell'alu) e viene richiesto come ingresso [Fase MEM/WB]

- Il forward è necessario solo se l'istruzione era una load e se il read non è x0. (Guarda le slide per ulteriori informazioni, questa parte va riscritta). Nel caso in cui ci siano sue hazard di questo tipo, bisogna prelevare il dato più recente.
- Control Hazard: l'azione di controllo dipende dall'istruzione precedente
 - E' necessario aggiungere hardware dedicato, ma devo comunque attendere un'istruzione (in teoria)
 - Posso prevedere la prossima istruzione, e fare tutto subito
 - Static branch: basato sul comportamento tipico
 - Dynamic branch: hardware dedicato che osserva quanto succede e migliora esaminando l'accuratezza delle predizioni (può cambiare in seguito a 1 o più predizioni errate)

Riassumendo, le pipeline migliorano la performance aumentando il throughput delle istruzioni, ma sono soggette agli hazard e la loro complessità è fortemente influenzata dall'ISA.

Organizzando il datapath in fasi, possiamo determinare quale fase ha la performance peggiore e stabilire il clock in base a quello. La latenza per eseguire un'istruzione non cambia da un design senza pipeline, ma le pipeline permettono il parallelismo nell'esecuzione delle istruzioni. I segnali di controllo, inoltre, vanno propagati sulle varie fasi (quindi devo aggiungere dei registri).

Nella rappresentazione multiciclo, metto in y le istruzioni e in x i tempi di clock e rappresento la pipeline con i componenti utilizzati per ogni istruzione. Ci si rende conto di Data Hazard quando la linea della dipendenza (che da dove si trova nella risorsa va al blocco) va all'indietro. In questi casi si possono quindi usare i forwarding.



Un exception è qualcosa di sollevato dal processore, come un opcode non definito oppure una chiamata al sistema, mentre un interrupt è una richiesta fatta da un dispositivo I/O, e entrambe possono causare problemi di performance seri. Quando si verifica un exception, si salva il PC dell'istruzione interrotta, si salva l'indicazione al problema e si passa al gestore dell'eccezione. Il gestore, o handler, è colui che si prende a carico il compito di determinare cosa fare per quell'eccezione. Le eccezioni rappresentano un'ulteriore control hazard se si verificano in una pipeline, e vanno trattate come si fa quando viene elaborata una predizione errata.

Un modo per aumentare l'ILP (instruction level parallelism) in un design a singolo processore è quello di replicare elementi della pipeline, in modo che in un solo ciclo di clock possano essere eseguite più istruzioni in un solo ciclo (il CPI si riduce), ma questo si può fare solo con istruzioni che non hanno dipendenze tra di loro.

Esistono due tipi di design: static multiple issue, il compilatore prende i gruppi di istruzioni e li ri-schedula in modo da eseguire più istruzioni possibili, in pratica raggruppa quelli senza dipendenze, e dynamic multiple issue, dove al posto del compilatore c'è l'hardware a farlo (può comunque esserci anche il compilatore in combinazione) in modo completamente dinamico. Entrambe i design prevedono un sistema speculativo per trovare la possibile istruzione successiva, e che avvia l'istruzione appena possibile verificando la validità dell'operazione: se la predizione è sbagliata, è previsto un meccanismo in grado di riparare all'errore. Nello static scheduling il compilatore deve rimuovere tutti gli hazard, come dipendenze con un pacchetto di istruzioni. Nei processori superscalari vengono impiegate le estensioni SIMD, che permettono di eseguire la stessa operazione su più operandi diversi.

Se il pipelining come idea è semplice, la realizzazione è molto complicata (gestione degli hazard) che può venir resa ancora più difficile da un ISA complesso.

GERARCHIE DI MEMORIA

Aumentando le prestazioni di un processore il collo di bottiglia si sposta altrove, come ad esempio la memoria.

1. Registri (pochi ma veloci)
2. Cache L1, piccole e straordinariamente veloci (ordine di grandezza 32/64kbyte), integrate nel processore
3. Cache L2, più lente delle precedenti ma più capienti (ordine di grandezza 1/2 MB), integrate nel processore
4. RAM (ordine di grandezza GB)
5. Dischi rigidi (ordine di grandezza TB)

Principio di località: I programmi accedono ad una piccola porzione della memoria a loro allocata in qualsiasi momento: gli elementi appena utilizzati possono essere riutilizzati a breve (località temporale), e anche i loro vicini (località spaziale).

Se ho bisogno di accelerare l'accesso ad un dato, lo sposto quindi in una memoria di gerarchia più grande in base al principio di località temporale. L'operazione di trasferimento, sebbene sia costosa, viene "ripagata" dal boost prestazionale che si ha quando si accede di nuovo al dato (mentre sposto su i dati scelti, sposto anche i dati vicini per il principio di località spaziale).

Quando avvio un programma, carico il programma in RAM e, a seconda dei dati che vengono usati, questi vengono caricati e scaricati dalle cache per velocizzare l'esecuzione. Le RAM moderne sono delle SDRAM (Synchronous Dynamic Random Access Memory), diverse dalla SRAM (Static Random Access Memory) che mantiene l'informazione finché non gli viene tolta la corrente (usa tanti transistor), quindi entrambe sono volatili. La DRAM non mantiene gli stati, in quanto viene implementata attraverso l'uso di un condensatore che può essere carico o scarico collegato ad un transistor che serve per far passare il dato (usa un solo transistor), e costa meno della SRAM (cache, registri). Perché è detta dinamica? Perché la carica nel condensatore tende a esaurirsi, quindi è necessario ogni tot leggere lo stato di carica e "rinfrescarlo". Sono dette sincrone in quanto hanno un clock che mi consente di inviare un indirizzo e di ricevere il contenuto delle celle a partire da quell'indirizzo (uno per ciclo di clock). Le RAM DDR (Double Data Rate) ad ogni ciclo di clock inviano segnali sia sul fronte di salita sia sul fronte di discesa.

Il core (nel risc-V) ha una granularità pari a 8 byte, ma le memorie?

Le informazioni dentro una cache sono rappresentate sotto forma di matrice, con una granularità di linea di cache (64 byte, ad esempio, quindi sono 500 righe se prevediamo una cache L1 da 32k). Il processore legge quindi i dati con la granularità sua (8 byte, ad esempio): se è nella cache, si verifica un cache hit, altrimenti è un cache miss, che richiede il caricamento da parte della cache di dati in ram in una riga di cache.

Una RAM è formata da n banchi di memoria, i quali a loro volta sono divisi in righe e colonne (ogni riga da, circa, 4k), e da buffer da una riga ciascuna per ogni banco. L'accesso ad una nuova riga di un banco RAM è molto onerosa, e i

buffer (row buffer) servono come cache (l'ultima riga alla quale si è fatto accesso viene lasciata lì), con un miss che costa centinaia di nanosecondi.

Le memorie di tipo flash non sono volatili, e sono 100x o 1000x volte più veloci degli hard disk. Possono venire implementate con porte NOR (memoria istruzioni in sistemi embedded) e NAND (dati), e sono soggette a rovinarsi dopo troppe letture e scritture.

Ci sono diversi modi per le cache per rendersi conto se il dato è già presente:

- Cache fully associative: necessario controllare ogni linea (lento)
- Direct mapped cache: gli indirizzi con una determinata caratteristica vanno sempre nella stessa linea (i dati nella cella di memoria il cui indirizzo termina per 101 va nella riga di cache 101). Di conseguenza, prima di fare qualsiasi altra cosa, va a vedere se il dato è già presente. Ma come faccio a capire se è quello giusto? Grazie all'indirizzo memorizzato insieme al dato (solo la parte che non è l'indirizzo della cache), chiamato TAG. Inoltre, in ogni blocco di memoria della cache è presente un bit di validità che rappresenta se il dato nella cache è stato inizializzato o meno (parte da 0 e viene portato a 1 quando ci viene caricato qualcosa). Come si trova a che blocco corrisponde un determinato indirizzo? Devo prima trovare il l'indirizzo del blocco (indirizzo/num_byte_per_blocco, per trovare il blocco sulla riga si utilizzano i byte meno importanti (offset)) e poi il block number (ind_blocco modulo blocchi_tot) che mi restituisce il numero della linea.

Blocchi più grandi riducono il miss rate, grazie alla località spaziale, ma in una cache dalla dimensione fissa a blocchi più grandi vuol dire che ce ne sono meno (più competizione = miss rate maggiore): i vantaggi offerti dalla località spaziale vengono sostituiti da altre pecche, e un modo per risolvere è utilizzare strategie di early restart e critical word first.

Riepilogando: se c'è un cache hit, tutto bene, ma se c'è un miss blocco la pipeline della cpu, prendo il blocco che si trova al livello di gerarchia inferiore. Se è un miss di istruzioni, riprendo il fetch, se è di dati termino l'operazione. Ma se modifichiamo valori? Dobbiamo anche aggiornare la memoria originale, altrimenti la cache e la memoria non concordano. Questo si può fare con il Write Through (aggiorna la memoria, ma è molto lento), meglio quindi tenere un buffer di scrittura, che attende la scrittura dei dati in memoria, con la cpu che stalla solo quando il buffer è pieno. Un'alternativa al Write Through è il Write Back, ovvero che scrive in memoria le modifiche solo quando il blocco in cui si contengono sta per venire sovrascritto, e viene usato un bit in più, il dirty bit, per segnalare se il dato è stato modificato. Che succede quando si verifica un write miss? Con il WT posso scegliere di riportarlo in cache oppure no, mentre con il write back viene tipicamente riportato in cache.

Con la split cache (cache sia per le istruzioni sia per i dati), i miss per le istruzioni sono molto pochi (meno dello 0.5%), mentre i data miss sono più frequenti.

Come già detto, noi utilizziamo la DRAM come memoria principale, dalla larghezza d'I/O definita. Se ci immaginiamo blocchi da 4 parole, sapendo che ci vuole un ciclo di bus per il trasferimento dell'indirizzo, 15 per l'accesso e 1 per il trasferimento dei dati, ci si mettono 65 cicli ($1+4*15+4*1$).

Il tempo utilizzato da un processore può essere diviso in cicli di esecuzione e cicli di stallo.

Un'alternativa alle cache Direct Mapped e fully associative è la Set associative a n vie, dove l'informazione va in n possibili locazioni ed ha bisogno solo di n comparatori (più economica). La cache diventa quindi alta alt/n di quanto avevamo prima con il directly mapped, ma con n cache affiancate (le way). Celle di memoria alla stessa altezza vengono chiamate set. E' quindi leggermente più costosa della direct mapped, ma ha molti meno cache miss.

Ma di quanta associatività c'è bisogno? Per trovarvi una risposta, simuliamo l'esecuzione di un programma su un sistema con 64kb di cache, con blocchi da 64 byte. Si osserva che la riduzione dei miss cala una volta superata la 2-Way, rendendola impercettibile.

In una direct mapped, non ci sono scelte di rimpiazzo, mentre in una set associative a n blocchi invece devo scegliere tra n blocchi. Una strategia, fattibile finché si opera al massimo con un 4-way, è scegliere il Least Recently Used (LRU) e scriverci sopra un valore nuovo. Quando ci sono molte vie, si può ricorrere al metodo Random, che sceglie in modo pseudo random una via (ovvero, se ci sono linee vuote va lì, altrimenti sovrascrive). Random e LRU, nel caso medio, sono equivalenti, ma nel caso peggiore la LRU è migliore: è facile mandare in crisi una LRU, ad esempio accedere ad un vettore di dimensione $n+1$, ed in questo è meglio la random.

DEPENDABILITY

La dependability è un valore che dipende da quanti errori si possono verificare in una determinata architettura, con il nostro sistema che oscilla tra service accomplishment (il servizio viene erogato normalmente) che, a causa di un failure, può portare ad un interruzione dei servizi che può essere risolta con un operazione di restore.

Esistono sistemi per la rilevazione di errore, come ad esempio quello dei bit di parità, e anche di autocorrezione. Un codice di autocorrezione è quello della distanza di Hamming, ovvero il numero di bit che cambia tra due pattern binari. Per usarlo, è necessario lasciare almeno 2 bit di spazio tra un dato e l'altro, che fornisce la rilevazione di un errore. Se se ne lasciano tre, permettono o la correzione dell'errore oppure la rilevazione di 2 errori. Questo vuol dire che le nostre sequenze da 4 bit possono rappresentare al massimo due numeri, quelle a 5 quattro e via dicendo. Come si calcola il codice di Hamming? Data una sequenza di valori, (ad esempio, una sequenza di 6 uno), metto dei bit riservati nelle posizioni multiple di 2, quindi abbiamo $p_1, p_2, 1, p_3, 1, 1, 1, p_4, 1, 1, 1$. Questi bit, che altro non sono che bit di parità, controllano la parità di gruppi di bit (p_1 controlla la parità dei valori terminanti con 0001, p_2 0010, p_3 0100, p_4 1000). Se vogliamo codificare il dato 1, avremo p_1, p_2, d , quindi 111.

MACCHINE VIRTUALI

Una macchina virtuale è un computer che opera all'interno di uno spazio simulato, che viene creato su una macchina mediante un programma chiamato hypervisor che virtualizza le risorse. L'utilizzo di VM permette un'isolamento migliorato, in quanto una macchina virtuale non *dovrebbe* essere in grado di accedere ad aree non consentite dall'hypervisor.

MEMORIA VIRTUALE

La memoria virtuale è un sistema per impiegare la memoria principale come cache di quella di massa, dando l'illusione di una memoria virtualmente illimitata. In questo tipo di memoria, ogni programma utilizza un indirizzo virtuale per accedere ai dati utilizzati più di frequente, con ogni area teoricamente protetta e inaccessibile per gli altri programmi. La conversione da indirizzo virtuale a fisico viene eseguita dal sistema operativo e dal processore. Un blocco di memoria virtuale è chiamato pagina, e i miss prendono il nome di page fault. In caso di page fault, la pagina va recuperata dal disco (per ridurre il tasso di page fault usiamo un piazzamento fully associative e algoritmi di rimpiazzo intelligenti). I dati relativi alla presenza delle pagine sono conservati nelle page tables (PTE), e per ridurre il page fault rate si sostituiscono le pagine meno utilizzate, e viene utilizzato un TLB per eseguire la traduzione rapida tra indirizzo virtuale e fisico (un TLB è una tabella con tag di validità, dirty, ref, il virtual page number e l'indirizzo). Se la pagina è in memoria, ma non nel TLB, allora viene caricata dal PTE e si riprova la lettura, mentre se non è in memoria viene eseguito dal OS il fetch della pagina.

Riassumendo, le memorie veramente veloci sono piccole e costose, quelle lente sono grandi e economiche. Per ottenere l'illusione di memorie veloci e grandi impieghiamo le cache e il concetto di gerarchia della memoria, legato al principio di località.

EXTRA – ALIAS DEI REGISTRI IN RISC-V

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5–7	t0–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller
f0–7	ft0–7	FP temporaries	Caller
f8–9	fs0–1	FP saved registers	Callee
f10–11	fa0–1	FP arguments/return values	Caller
f12–17	fa2–7	FP arguments	Caller
f18–27	fs2–11	FP saved registers	Callee
f28–31	ft8–11	FP temporaries	Caller